شوگاریستا · مستندات فنی

راهنمای استقرار ایمیج‌محور

از لپ‌تاپ تا سرور — یک بسته، یک SFTP، بدون git، بدون build روی سرور، بدون هیچ رجیستری واسط

نسخه ۱.۱ ۱۷ آبان ۱۴۰۵ مخاطب: مدیر سیستم / توسعه‌دهنده

فهرست

1. تصویر کلی
2. فایل‌های مربوط به دیپلوی
3. یک‌بار تنظیم کنید: هدف SFTP
4. اولین Release روی لپ‌تاپ
5. آماده‌سازی سرور (فقط یک‌بار)
6. چرخهٔ روزمره
7. برگشت به نسخهٔ قبلی
8. بکاپ خودکار
9. نکات و تله‌ها
10. چک‌لیست هر دیپلوی
11. چرا این روش امن است

۰ تصویر کلی

در این روش، ایمیج‌ها روی **لپ‌تاپ خودتان** ساخته می‌شوند، در یک بستهٔ واحد (`dist/sugarista-<tag>.tar.gz`) بسته‌بندی می‌شوند و با یک **SFTP** مستقیم روی سرور می‌روند. سرور فقط بسته را `docker load` می‌کند و اجرا. **هیچ رجیستری واسطی نیست**: نه Docker Hub، نه GHCR — ایمیج هیچ‌وقت از دستگاه شما خارج نمی‌شود مگر به سرور خودتان.



نکتهٔ کلیدی

دیتابیس و فایل‌های آپلودی در **named volume**هستند. با هر دیپلوی، کانتینرها عوض می‌شوند ولی volumeها دست‌نخورده می‌مانند. تنها دستوری که می‌تواند دیتا را پاک کند `docker compose down -v` است — هرگز آن را اجرا نکنید.

۱ فایل‌های مربوط به دیپلوی

فایل	کجا اجرا می‌شود	کار
`docker-compose.prod.yml`	سرور	استک فقط-ایمیج: بدون `build`، بدون bind mount، بدون هیچ رجیستری
`deploy/nginx/Dockerfile`	لپ‌تاپ	کانفیگ nginx را داخل ایمیج می‌پزد
`deploy/.env-upload.example`	لپ‌تاپ	قالب `SERVER_HOST` و `SERVER_PATH` — کپی کنید به `deploy/.env-upload` (در git نادیده گرفته می‌شود)
`scripts/release.sh`	لپ‌تاپ	۴ ایمیج را برای `linux/amd64` می‌سازد و در `dist/sugarista-<tag>.tar.gz` بسته می‌کند
`scripts/upload-images.sh`	لپ‌تاپ	بسته را با SFTP به سرور می‌فرستد؛ با `WITH_FILES=1` فایل compose و scripts را هم همگام می‌کند
`scripts/ship.sh`	لپ‌تاپ	کل زنجیره با یک دستور: ساخت → آپلود → دیپلوی ریموت
`scripts/server-deploy.sh`	سرور	`docker load` → بکاپ → up → health → smoke test
`scripts/backup-db.sh`	هر دو	بکاپ فشردهٔ دیتابیس با نگه‌داری N نسخهٔ آخر
`scripts/restore-db.sh`	هر دو	بازگردانی کامل دیتابیس از بکاپ

تفاوت‌های مهم با فایل توسعه

- همهٔ سرویس‌ها `image:` دارند و هیچ‌کدام `build:` ندارند.
- نام‌های ایمیج دیگر پیشوند رجیستری ندارند: `sugarista-api:<tag>` — دقیقاً همان چیزی که بسته در خود دارد.
- `public-ui` دیگر پوشهٔ `Sugarista-Theme` را mount نمی‌کند؛ فایل‌ها داخل ایمیج هستند.
- `proxy` دیگر `deploy/nginx/default.conf` را mount نمی‌کند؛ کانفیگ داخل ایمیج است.
- `name: sigarista` قفل شده تا نام volume هرگز عوض نشود.

۲ یک‌بار تنظیم کنید: هدف SFTP

اسکریپت آپلود بدانند بسته را به کجا بفرستد — فقط یک‌بار لازم است.

۲-۱) کلید SSH

اگر هنوز کلیدتان را روی سرور کپی نکرده‌اید:

```
ssh-keygen -t ed25519          # اگر کلید ندارید
ssh-copy-id root@sugarista-web
```

`sugarista-web` یک نام مستعار در `~/.ssh/config` است:

```
Host sugarista-web
    HostName 203.0.113.10
    User root
    Port 22
```

۲-۲) فایل تنظیمات آپلود

```
cp deploy/.env-upload.example deploy/.env-upload
```

و در آن فقط این دو مقدار:

```
SERVER_HOST=root@sugarista-web
SERVER_PATH=/opt/sugarista/Sigarista
```

این فایل git-ignored است. اگر نخواهید فایل بسازید، می‌توانید متغیرها را موقتاً بدهید:

`SERVER_HOST='root@203.0.113.10' ./scripts/upload-images.sh`

۳ اولین Release روی لپ‌تاپ

```
cd ~/Desktop/Sugarista-Project
./scripts/release.sh
```

خروجی مورد انتظار:

```
==> Building sugarista-api (linux/amd64)
==> Building sugarista-admin (linux/amd64)
==> Building sugarista-theme (linux/amd64)
==> Building sugarista-proxy (linux/amd64)
==> Packing images into dist/sugarista-2026.11.09-1200.tar.gz

Bundle   : dist/sugarista-2026.11.09-1200.tar.gz (xxx MB)
Tag      : 2026.11.09-1200  (also tagged :latest)
Platform : linux/amd64

Next step — upload via SFTP to the server:
  ./scripts/upload-images.sh
```

بستهٔ `.tar.gz` شامل هر ۴ ایمیج است؛ لایه‌های مشترک مصالحه (dedup) می‌شوند و حجم را کنترل می‌کند. سرور برای راه‌اندازی به این یک فایل بسنده می‌کند.

> **زمان ساخت**
>
> بار اول برای ایمیج API روی مک حدود ۴ تا ۵ دقیقه طول می‌کشد (شبیه‌سازی معماری amd64). بارهای بعد به‌خاطر کش بسیار سریع‌تر است.

گزینه‌های مفید

```
IMAGE_TAG=v1.2 ./scripts/release.sh                # تگ دلخواه
BUILD_ONLY=1 ./scripts/release.sh                  # فقط build، بدون بسته‌بندی
TARGET_PLATFORM=linux/arm64 ./scripts/release.sh   # اگر سرور ARM بود
```

> **معماری**
>
> سرور x86_64 است، پس همیشه `TARGET_PLATFORM=linux/amd64` (پیش‌فرض) درست است. ایمیج arm64 روی سرور اجرا نمی‌شود.

۴ آماده‌سازی سرور (فقط یک‌بار)

۴-۱) نام volume فعلی را بررسی کنید

این مرحله حیاتی است؛ اگر نام عوض شود، استک جدید یک دیتابیس **خالی** می‌سازد.

```
ssh sugarista-web 'docker volume ls | grep postgres-data'
```

باید چیزی مثل `sigarista_postgres-data` ببینید؛ یعنی پیشوند پروژه `sigarista` است که با `name: sigarista` در فایل جدید هم‌خوان است. اگر پیشوند چیز دیگری بود، در `.env` سرور اضافه کنید:

```
COMPOSE_PROJECT_NAME=<همان-پیشوند>
```

۴-۲) یک بکاپ اضافه بگیرید

```
ssh sugarista-web
cd /opt/sugarista/Sigarista
docker compose exec -T postgres pg_dump -U sugarista -d sugarista \
  --no-owner --no-privileges | gzip > /root/pre-image-deploy.sql.gz
ls -lh /root/pre-image-deploy.sql.gz
```

۴-۳) فایل‌های پایه را با همان SFTP بفرستید

روی لپ‌تاپ — بعد از اولین release:

```
cd ~/Desktop/Sugarista-Project
WITH_FILES=1 ./scripts/upload-images.sh
```

این کار هم بستهٔ ایمیج را می‌فرستد و هم `docker-compose.prod.yml` و پوشهٔ `scripts/` را. از این به بعد فقط خود بسته‌ها جابه‌جا می‌شوند.

۴-۴) استک قدیمی را خاموش کنید — بدون `-v`

```
ssh sugarista-web
cd /opt/sugarista/Sigarista
docker compose down
```

۴-۵) اولین دیپلوی ایمیجی

```
cd /opt/sugarista/Sigarista
chmod +x scripts/*.sh
./scripts/server-deploy.sh
```

خروجی موفق:

```
==> Target image tag: 2026.11.09-1200
==> Backing up database
==> Loading images from sugarista-2026.11.09-1200.tar.gz
==> Applying changes
==> Waiting for API health
==> Smoke test
  /health  -> 200
  /admin/  -> 200
  /        -> 200
==> Deployed image tag 2026.11.09-1200 successfully.
```

از این لحظه، سرور فقط ایمیج اجرا می‌کند. سورس‌کد قدیمی را می‌توانید پاک کنید — به‌جز این‌ها:

```
/opt/sugarista/Sigarista/
├── .env                    ← رمزها، دست نزنید
├── docker-compose.prod.yml
├── scripts/
├── dist/                   ← بسته‌های ایمیج (برای rollback)
└── backups/
```

۵ چرخهٔ روزمره

یکی از دو مسیر را انتخاب کنید:

گزینهٔ A — یک دستور (پیشنهادی)

روی لپ‌تاپ، کل زنجیره: ساخت ← آپلود ← دیپلوی ریموت:

```
cd ~/Desktop/Sugarista-Project
./scripts/ship.sh
```

خروجی هر سه مرحله را پشت سر هم نشان می‌دهد و smoke test سرور را چک می‌کند.

```
SHIP_NO_DEPLOY=1 ./scripts/ship.sh      # فقط ساخت + آپلود، دیپلوی را خودتان بعداً
SKIP_BUILD=1 ./scripts/ship.sh          # از بستهٔ موجود در dist/ را بفرست
```

گزینهٔ B — مرحله‌به‌مرحله

```
# لپ‌تاپ
./scripts/release.sh          # ساخت + بسته‌بندی
./scripts/upload-images.sh    # SFTP

# سرور (از لپ‌تاپ یا با یک خط ssh)
ssh sugarista-web "cd /opt/sugarista/Sigarista && ./scripts/server-deploy.sh"
```

۶ برگشت به نسخهٔ قبلی (Rollback)

هر بسته یک تگ نسخه‌دار در خود دارد؛ سرور بسته‌های قدیمی را در `dist/` نگه می‌دارد:

```
cd /opt/sugarista/Sigarista
IMAGE_TAG=2026.11.08-0900 ./scripts/server-deploy.sh
```

اگر بستهٔ آن تگ روی سرور حذف شده باشد، اول دوباره آپلودش کنید:
`IMAGE_TAG=2026.11.08-0900 ./scripts/upload-images.sh` (لپ‌تاپ) — بسته‌های قبلی در `dist/` لپ‌تاپ هم مانده‌اند.

اگر مایگریشن اجرا شده بود

اول کد را برگردانید. اگر مشکل ادامه داشت، دیتابیس را از بکاپ بازگردانید:
`./scripts/restore-db.sh backups/sugarista-2026xxxx-xxxxxx.sql.gz`

۷ بکاپ خودکار روزانه

```
crontab -e
```

```
0 3 * * * cd /opt/sugarista/Sigarista && COMPOSE_FILE=/opt/sugarista/Sigarista/docker-compose.prod.yml ./scripts/backup-db.sh >> /var/log/sugarista-backup.log 2>&1
```

و کپی offsite:

```
0 4 * * * rclone copy /opt/sugarista/Sigarista/backups/ remote:sugarista-backups/ --max-age 48h
```

قانون طلایی بکاپ

بکاپی که یک‌بار restore نشده باشد، بکاپ نیست. ماهی یک‌بار روی یک محیط تست، restore را تمرین کنید.

۸ نکات و تله‌ها

موضوع	نکته
معماری	سرور x86_64 است؛ همیشه از `linux/amd64` استفاده کنید
نام پروژه	`name: sigarista` قفل شده؛ پوشه را جابه‌جا نکنید.
`down -v`	هرگز. تنها دستوری است که دیتا را می‌کشد.
تغییر تم	چون bind mount حذف شد، نیاز به release جدید دارد.
تغییر nginx	کانفیگ داخل ایمیج proxy پخته می‌شود؛ بعد از تغییر، release بزنید.
`.env`	هرگز داخل ایمیج نمی‌رود و با دیپلوی عوض نمی‌شود.
مایگریشن	API روی استارت خودش `MigrateAsync()` را اجرا می‌کند.
نام بسته	اسکریپت‌ها تگ را **از نام فایل** می‌خوانند — بسته‌ها را تغییر نام ندهید.
دسترسی سرور	دیگر هیچ `docker login` لازم نیست؛ همه‌چیز با کلید SSH می‌گردد.
فضای دیسک	هر بسته چند صد مگابایت است؛ بعد از دیپلوی موفق می‌توانید بسته‌های خیلی قدیمی را از `dist/` سرور پاک کنید (حداقل ۲ تگ اخیر را نگه دارید).
رمز ادمین	با اولین seed ست می‌شود و با دیپلوی عوض نمی‌شود.

۹ چک‌لیست هر دیپلوی

لپ‌تاپ

- ☐ `dotnet test` سبز است
- ☐ `npm run build` در `Sugarista-Admin` بدون خطاست
- ☐ `./scripts/ship.sh` (یا release + upload) بدون خطا تمام شد و تگ نسخه را یادداشت کردم

سرور

- ☐ `df -h` حداقل ۲ تا ۳ گیگ فضای آزاد نشان می‌دهد
- ☐ `./scripts/server-deploy.sh` هر سه endpoint را ۲۰۰ داد
- ☐ یک‌بار `/admin/` را باز کردم و لاگین گرفتم

۱۰ چرا این روش امن است

- **دیتابیس و آپلودها** در volume هستند و اصلاً به کد کاری ندارند.
- **هیچ رجیستری ثالثی** در کار نیست — ایمیج فقط بین لپ‌تاپ و سرور خودتان جابه‌جا می‌شود، روی کانال SSH رمزنگاری‌شده.
- **سرور هیچ‌وقت کامپایل نمی‌کند** → نه NuGet قطع می‌شود، نه npm، نه رم سرور.
- **هر نسخه یک بستهٔ تغییرناپذیر** است → rollback دقیقاً همان چیزی را برمی‌گرداند که قبلاً کار می‌کرد.
- `.env` **و رمزها** هرگز وارد ایمیج یا ریپو نمی‌شوند.
- **بکاپ خودکار** قبل از هر دیپلوی گرفته می‌شود.

برگهٔ مرجع سریع

می‌خواهم...	دستور
کل زنجیره را با یک دستور بزنم (ساخت + آپلود + دیپلوی)	`./scripts/ship.sh`
نسخهٔ جدید بسازم و بسته‌بندی کنم	`./scripts/release.sh`
فقط بسته را بفرستم	`./scripts/upload-images.sh`
روی سرور مستقر کنم	`ssh sugarista-web "cd /opt/sugarista/Sigarista && ./scripts/server-deploy.sh"`
به نسخهٔ قبل برگردم	`IMAGE_TAG=<old> ./scripts/server-deploy.sh` (سرور)
بکاپ بگیرم	`./scripts/backup-db.sh` (سرور)
بکاپ را برگردانم	`./scripts/restore-db.sh backups/<file>.sql.gz` (سرور)
لاگ API را ببینم	`docker compose -f docker-compose.prod.yml logs -f api`
وضعیت را ببینم	`docker compose -f docker-compose.prod.yml ps`
